



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch

Vedoucí práce

Ing. Martin Dostal, Ph.D.

© Bc. Milan Janoch, 2026.

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

JANOCH, Bc. Milan. *Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)*. Plzeň, 2026. Diplomová práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Ing. Martin Dostal, Ph.D.

Podklad pro zadání DIPLOMOVÉ práce studenta

Jméno a příjmení: **Bc. Milan JANOCH**

Osobní číslo: **A24N0104P**

Adresa: **V Brance 20, Přeštice, 33401 Přeštice, Česká republika**

Téma práce: **Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)**

Téma práce anglicky: **Automatic generation of end-to-end tests using large language models (LLM)**

Jazyk práce: **Čeština**

Související osoby: **Ing. Martin Dostal, Ph.D. (Vedoucí)**

Katedra informatiky a výpočetní techniky

Zásady pro vypracování:

1. Prostudujte současné možnosti generování kódu pomocí velkých jazykových modelů (LLM), se zvláštním zaměřením na oblast automatizovaného testování softwaru.
2. Analyzujte strukturu uživatelských příběhů a specifikací běžně používaných v nástrojích pro řízení vývoje softwaru a identifikujte klíčové prvky využitelné pro generování testovacích scénářů.
3. Navrhněte architekturu systému pro automatické generování end-to-end (E2E) testů z uživatelských příběhů s využitím LLM.
4. Implementujte funkční prototyp navrženého řešení využívající vybraný LLM a vhodný framework pro E2E testování.
5. Otestujte funkčnost systému na vzorových scénářích a vyhodnoťte kvalitu generovaných testů.
6. Kriticky zhodnoťte přínosy a limity řešení a navrhněte možnosti dalšího rozšíření.

Seznam doporučené literatury:

Dodá vedoucí diplomové práce

Podpis studenta:

Datum:

Podpis vedoucího práce:

Datum:

Prohlášení

Prohlašuji, že jsem tuto diplomovou práci vypracoval samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Tato práce nebyla využita k získání jiného nebo stejného akademického titulu.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Západočeská univerzita v Plzni má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Plzni dne 01. března 2026

.....

Bc. Milan Janoch

V textu jsou použity názvy produktů, technologií, služeb, aplikací, společností apod., které mohou být ochrannými známkami nebo registrovanými ochrannými známkami příslušných vlastníků.

Abstrakt

xxx

Abstract

xxx

Klíčová slova

xxx • xxxx • xxx

Obsah

1	Úvod	2
2	Analýza současného stavu	3
2.1	Agilní vývoj a specifikace požadavků	3
2.1.1	Agilní metodiky	3
2.1.2	Uživatelské příběhy	4
2.1.3	Akceptační kritéria	5
2.2	Automatizované testování softwaru	6
2.2.1	Metodika a pyramida testování	6
2.2.2	E2E testování	8
2.2.3	Přehled frameworků pro webovou automatizaci	9
2.2.4	Výzvy automatizace v agilním prostředí	10
2.3	Velké jazykové modely ve vývoji softwaru	11
2.4	Agentní systémy a autonomní testování	11
3	Závěr	12
	Přehled zkratk	13
A	Uživatelská dokumentace	14
	Bibliografie	15
	Seznam obrázků	18
	Seznam tabulek	19
	Seznam výpisů	20

V dnešním světě, kdy se software i díky nástupu umělé inteligence vyvíjí rychleji než kdy dříve, je zajištění kvality a spolehlivosti aplikací klíčovým faktorem pro úspěch. Testování hraje zásadní roli při odhalování chyb, zajištění důvěry uživatelů a zvyšování kvality softwaru. S nástupem velkých jazykových modelů (LLM) se objevují nové příležitosti pro automatizaci generování testů [1], což může výrazně zefektivnit proces testování a snížit náklady spojené s manuálním vytvářením testovacích scénářů. Hranice možností se stále posouvají nejen co se týče výkonností jednotlivých modelů (např. známých GPT, Claude) a společností přicházejících na trh, ale i co se týče přístupů k využití těchto modelů pro generování testů. I přes tento inovační potenciál je ale třeba myslet na rizika a výzvy spojené s rozšířeným využíváním LLM - a to nejen z pohledu softwarového (např. riziko halucinací, nedostatečná kvalita generovaných testů), ale i na cenovou a etickou stránku věci (např. náklady na využívání LLM, otázka ochrany dat a soukromí).

Cílem této diplomové práce je prozkoumat možnosti generování kódu pomocí velkých jazykových modelů v rámci automatizovaného testování a implementovat funkční prototyp, který má potenciál výrazně urychlit a zefektivnit proces testování softwaru pomocí end-to-end testů. Aby aplikace byla prakticky použitelná, bude zaměřená na generování testů na základě uživatelských příběhů, jež jsou běžně využívány v agilním vývoji softwaru [2] a poskytují přehledný a srozumitelný popis požadavků z pohledu uživatele. To umožní rozšířit možnosti psaní spustitelných testů i do oddělení jako je QA (Quality Assurance), které nemusí mít hluboké technické znalosti, ale jsou klíčové pro zajištění kvality softwaru. Důraz bude kladen na kvalitu vygenerovaných testů, jejich schopnost odhalovat chyby a přínos pro zefektivnění procesu testování.

Analýza současného stavu

2

2.1 Agilní vývoj a specifikace požadavků

2.1.1 Agilní metodiky

IT je jedním z nejrychleji se rozvíjejících odvětví a to nejen z pohledu technologií, ale i přístupu k vývoji softwaru. Jak [3] uvádí, tento aspekt vedl k vývoji metodik (např. agilní metodiky či vodopádový model), které navrhují řízení v závislosti na vlastnostech produktu, organizačních strukturách či dynamickém prostředí. Tradiční konzervativnější vodopádový model (populární např. ve stavebnictví) se zaměřuje na důkladné plánování [4] a znalost požadavků před zahájením vývoje [5]. V rychle se měnícím prostředí je ale tento přístup často neefektivní, protože požadavky ze strany uživatelů se mohou rychle měnit a není možné je znát všechny předem. Tento problém řeší agilní metodiky, které kladou důraz na flexibilitu - díky tomu, že fungují na tzv. iterativním principu (software se vyvíjí v krátkých cyklech často přezdívané jako sprinty), umožňují průběžné přizpůsobování se měnícím se požadavkům a rychlejší dodávání funkčních verzí softwaru [5].

Nejznámějším a současně nejrozšířenějším přístupem k agilnímu vývoji je Scrum. Scrum tým se zpravidla skládá z následujících rolí [4, 6, 7, 8]:

- **Vývojáři (Developers)** - jsou zodpovědní za vývoj softwaru a implementaci požadovaných funkcionalit (featur).
- **Scrum Master** - zajišťuje dodržování Scrum procesů, podporuje tým při práci a pomáhá odstraňovat překážky, které by mohly bránit efektivnímu vývoji.
- **Product Owner** - je zodpovědný za správu produktového backlogu, komunikaci s vývojovým týmem a zainteresovanými stranami (stakeholdery) a za zajištění toho, aby vývoj směřoval k naplnění potřeb zákazníka.

Hlavní náplní práce Product Ownera je správa backlogu a příprava uživatelských příběhů (často známý pod rozšířenějším anglickým pojmem *user stories*), které specifikují požadované funkcionality z pohledu uživatele [8].

2.1.2 Uživatelské příběhy

User story je nejpoužívanějším artefaktem v agilním vývoji [9] - jedná se o krátký, semi-strukturovaný popis požadavků napsán v přirozeném jazyce. Na obrázku 2.1 je uvedena standardní šablona, podle níž se uživatelské příběhy píší [9, 10, 11].

Standard User Story Template	
As a	<type of user / persona>
I want	<to perform some action / goal>
So that	<some value / benefit is achieved>

Obrázek 2.1: Standardní šablona uživatelského příběhu.

Každá user story by měla být napsána z pohledu konkrétního uživatele (*persona*), s jasně stanoveným cílem (*goal*) a dosaženou hodnotou (*value*). Hodnota je dosažena pouze tehdy, pokud byl plánovaný cíl úspěšně splněn [11].

Kvalita napsaných user storek je klíčová nejen pro porozumění vývojářů při implementaci, ale také pro následné testování a ověření, že plánovaný cíl byl skutečně dosažen. Jednou z metod, která pomáhá jasně definovat kritéria pro úspěšné splnění user story, je INVEST [12]. V tabulce 2.1 jsou uvedeny jednotlivé principy INVEST, které by měly být dodržovány pro psaní kvalitních uživatelských příběhů.

Princip	Popis
Independent	User story by měla být co nejvíce nezávislá na ostatních stories.
Negotiable	Požadavky by měly být otevřené pro diskusi a případné úpravy.
Valuable	Splnění user story musí přinést jasnou hodnotu koncovému uživateli.
Estimable	Musí být možné odhadnout náročnost implementace (např. čas nebo potřebné zdroje).
Small	User story by měla být dostatečně malá, aby bylo možné ji dokončit během jednoho sprintu.
Testable	Musí obsahovat definovaná akceptační kritéria, podle kterých lze ověřit splnění požadavku.

Tabulka 2.1: INVEST principy pro tvorbu kvalitních user stories [13].

2.1.3 Akceptační kritéria

Zatímco user story charakterizují potřeby zákazníka, akceptační kritéria (často označována zkratkou ACC z anglického Acceptance Criteria) jasně popisují Definition of Done (zkráceně DoD) pro konkrétní user story [14] co je nutno dodržet a udělat, aby byla user story považována za splněnou. Týkají se zejména funkcionality (tlačítko dělá to co má), kvalitativních charakteristik (systém reaguje do X milisekund) a obchodní logiky [15].

Akceptační kritéria mohou být zapsána různou formou - ať už prostý seznam, kde každý bod představuje právě jedno kritérium, nebo formou scénářů, které jsou popisovány pomocí jazyka Gherkin. Tento způsob testování (kdy jsou nejprve napsány testovací scénáře v přirozeném jazyce a následně se implementují samotné testy) se nazývá Behavior-Driven Development (BDD) [16].

Feature: User Login

Scenario: Successful login with valid credentials

Given the user is on the “/login” page

When the user enters “user@example.com” into the email field

And the user enters “password123” into the password field

And the user clicks the “Login” button

Then the user should be redirected to the “/dashboard” page

And a welcome message “Welcome back” should be displayed

Obrázek 2.2: Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.

Hlavní motivací pro využití Gherkin scénářů je vysoká abstrakce - scénáře jsou psány v přirozeném jazyce, což umožňuje jednodušší komunikaci mezi technicky zdatnými i netechnickými členy týmu a zároveň poskytují jasný popis požadavků a očekávaného chování [17].

Pro prostý seznam akceptačních je důležité, aby používal jasný a konzistentní jazyk a zaměřoval se hlavně na testovatelnost, nikoliv na formát zápisu [15]. Ukázka takového seznamu je uvedena na obrázku 2.3 - každý bod by měl být jasně identifikovatelný a obsahuje jedno kritérium, které lze ověřit.

Acceptance Criteria for US_01: User Login

- **ac_01 (Success):** System grants access to the dashboard when valid credentials (email and password) are provided.
- **ac_02 (Success):** Navigation to the login page is possible via the “Sign In” button on the landing page.
- **ac_03 (Failure):** System displays an error message “Invalid credentials” if the password does not match the stored hash.
- **ac_04 (Failure):** Account is temporarily locked after five consecutive failed login attempts.
- **ac_05 (Robustness):** Input fields prevent the submission of SQL injection patterns or excessively long strings.

Obrázek 2.3: Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu s unikátní identifikací.

V praxi bývají akceptační kritéria (spolu s uživatelskými příběhy) nejčastěji obsažena v ALM nástrojích (Application Lifecycle Management) jako jsou Jira, Azure DevOps, Rally či GitHub Projects. Tyto nástroje nevyžadují specifický formát zápisu, ale umožňují flexibilitu - ať už jde o prostý seznam, Gherkin scénáře nebo kombinaci obojího. Např. studie [18] zaměřená na generování akceptačních testů vycházela z běžných user stories a jejich textových popisů, z něhož následně generovala Gherkin scénáře, z nichž až poté byly implementovány automatizované testy.

2.2 Automatizované testování softwaru

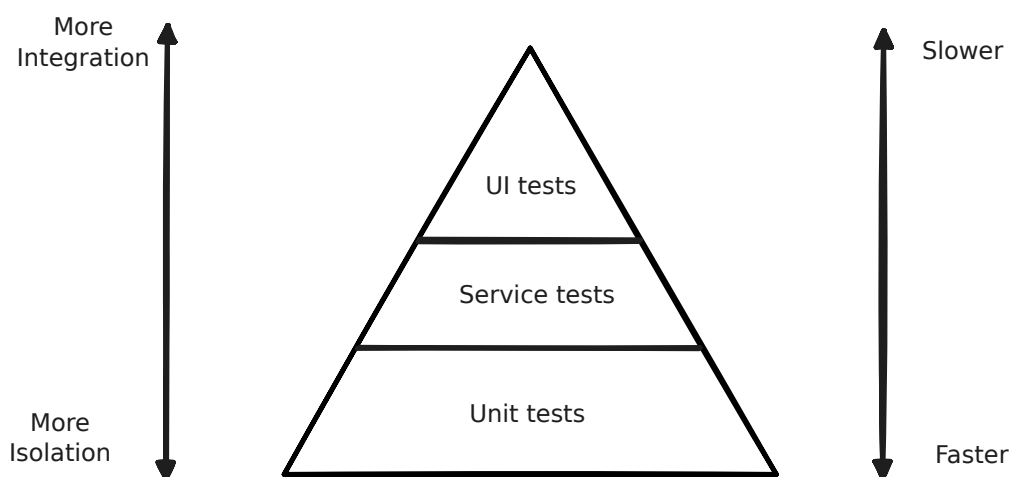
2.2.1 Metodika a pyramida testování

Softwarové testování lze definovat jako hledání chyb či chybějících požadavků v softwaru [19]. Dá se na něj nahlížet i z pohledu snižování rizika, což vede v posílení důvěry uživatelů a zlepšení kvality softwaru. Jak také studie [19] uvádí, testování je důležitou součástí QA - čím dříve jsou chyby odhaleny, tím levnější je jejich oprava. Testování lze rozdělit do několika tříd, jež jsou znázorněny v tabulce 2.2, která vychází z knihy [20].

Název testu	Popis
Testování podle scénářů	Tester dle předpřipravených scénářů manuálně prochází aplikaci a kontroluje očekávané chování jednotlivých testovacích kroků.
Průzkumné testování	Tester bez předem známých scénářů prozkoumává aplikaci a hledá chyby.
Regresní testy	Ověřují, zda nově přidané featury nezhoršily existující funkce.
Jednotkové (unit) testy	Automatizované skripty, které izolovaně ověřují funkčnost nejmenších částí zdrojového kódu (funkce, metody).
Beta testy	Poskytnutí aplikace omezené skupině uživatelů.
Smoke testy	Rychlé počáteční testy ověřující sestavení aplikace a její stabilitu.
Konfirmační testy	Potvrzení, že aplikace po odstranění defektu funguje správně.

Tabulka 2.2: Druhy testů dle [20].

Na obrázku 2.4 je uvedena pyramida testování, která znázorňuje důležitost jednotlivých druhů testů v rámci automatizace testování [21].



Obrázek 2.4: Pyramida testování Mika Cohna (vektorový formát).

Jednotkové testy jsou nejvíce izolované, píšou se rychleji, a tudíž jsou i levnější. Naproti tomu UI testy (User Interface - testy uživatelského rozhraní - např. end-to-end) vyžadují větší integraci a jejich vývoj je zpravidla dražší (testy se píšou déle a déle běží). Přestože jsou end-to-end testy dražší, představují nedílnou součást celé automatizace - bez těchto testů není zaručeno, že komponenty/moduly systému spolu správně komunikují a dávají tak koncovému uživateli očekávaný stav.

Při psaní testů se objevují speciální případy testů - a to testy chybně napsané či nestabilní testy. Jedná se o následující termíny [20]:

- **Falešně pozitivní (False Positive)** - Test, který selže, přestože testovaná funkcionality funguje správně.
- **Falešně negativní (False Negative)** - Test, který projde, i když testovaná funkcionality nefunguje správně.
- **Křehké (Fragile)** - Test, který selže kvůli změně v implementaci, přestože testovaná funkcionality zůstala nezměněna (např. změna názvu HTML elementu).
- **Nestabilní (Flaky)** - Test, který se chová nedeterministicky - může selhat nebo projít bez jakýchkoliv změn v kódu, což ztěžuje identifikaci skutečných problémů (např. problémy se síťovou komunikací).

Těmto problémům se při psaní testů snaží vývojáři předcházet - a to např. důrazem na determinismus testů, kontrolou správnosti testovaného chování na Code Review nebo využíváním nástrojů pro správu testů, které pomáhají identifikovat a řešit nestabilní testy.

2.2.2 E2E testování

End-to-end testování (zkráceně E2E) je typ testování, který ověřuje kompletní funkčnost systému z pohledu koncového uživatele. Využívá se přístupu tzv. metody **black-box** (černé skříňky), kdy tester nemá přístup k implementaci či struktuře systému, ale testuje systém jako jeden celek [20, 22]. V případě webových aplikací je možné nahlížet na způsoby tvorby end-to-end testů dvěma způsoby [22]:

- **Capture-Replay** - Tester nejdříve manuálně prochází aplikaci, zaznamenává své akce a následně se tyto akce převedou na automatizovaný skript.
- **Programmable Web Testing** - Testy se rovnou píšou jako zdrojový kód, který využívá frameworky pro webovou automatizaci.

Hlavní výhodou přístupu **Capture-Replay** je rychlost - testy lze vytvořit velmi rychle, nevýhodou je ale nízká kvalita a nestabilita testů, které jsou často náchylné k falešně negativním výsledkům (např. když se HTML elementy změní, program funguje stále správně, ale test selže).

Obě tyto perspektivy potřebují lokalizovat a interagovat s HTML elementy. Interakce s HTML elementy lze rozdělit do tří kategorií [22]:

- **Coordinate-based** - Využívání [X,Y] souřadnic pro interakci s elementy.
- **DOM-based** - Využití Document Object Modelu (DOM)/HTML struktury.
- **Vizuální** - Využití rozpoznání obrazu pro určení elementů na obrazovce.

Zatímco první kategorie je označována jako zastaralá [22], DOM-based přístup je náchylný na drobné změny v HTML struktuře, které mohou způsobit selhání testů a vyžadují častou údržbu. Vizuální přístup sice dokáže testovat to, co uživatel skutečně vidí, ale je náchylný na změny v rozložení stránky a režie spojená s rozpoznáváním obrazu může být vysoká. Právě snaha o řešení těchto problémů vedla k zrodu mnoha testovacích frameworků pro webovou automatizaci, které se snaží zjednodušit a zefektivnit tvorbu end-to-end testů.

2.2.3 Přehled frameworků pro webovou automatizaci

V oblasti webové automatizace existuje velké množství frameworků, které se navzájem liší svými vlastnostmi, podporovanými jazyky a přístupy k testování. Zde je uveden přehled těch nejpopulárnějších a nejčastěji používaných,

2.2.3.1 Selenium (WebDriver)

Jeden z nejrozšířenějších a nejstarších nástrojů. Podporuje širokou škálu jazyků (Java, Python, C#, JavaScript) a prohlížečů prostřednictvím WebDriveru [23]. Díky rozšířené komunitě, množství dostupných zdrojů a existujících knihoven je Selenium stále velmi populární volbou. Nevýhodou je složitější prvotní konfigurace a manuální správa testů, která může (při nesprávném použití) vést k nestabilním testům.

2.2.3.2 Robot Framework

Open-source framework založený na Pythonu, který využívá klíčová slova pro psaní testů (tzv. Keyword-Driven Testing). Architektura využívá knihovny SeleniumLibrary pro webovou automatizaci [24]. Silnou stránkou je jednoduché psaní testů - testy jsou psány čitelně v přirozeném jazyce, díky čemuž jim porozumí i netechničtí členové týmu. Se zvyšující se komplexností webových aplikací a testů může být nevýhodou nižší flexibilita a nutnost používat externí knihovny pro pokročilé funkce.

2.2.3.3 Playwright

Moderní testovací framework vyvinutý společností Microsoft. Podporuje více jazyků (JavaScript/TypeScript, Python, Java, .NET) a nabízí robustní API pro interakci

s webovými stránkami [25]. Výhodou je rychlost testů a podpora pro více prohlížečů (Firefox, WebKit, Chromium) bez nutnosti externích driverů. Klíčovou funkcí je také `Auto-waiting`, který pomáhá eliminovat křehkost testů. Nevýhodou je menší komunita a méně dostupných zdrojů, jelikož se jedná o relativně nový nástroj.

2.2.3.4 Cypress

Nástroj zaměřený na testování moderních webových aplikací, který podporuje pouze JavaScript a TypeScript. Běží přímo v prohlížeči [26], což urychluje vývoj testů a jejich případné debugování. Disponuje také funkcí `Time Travel`, která umožňuje procházet test krok po kroku a vidět stav aplikace v každém kroku. Nevýhodou je omezená podpora napříč jazyky, prohlížeči a testování vícero tabů.

2.2.4 Výzvy automatizace v agilním prostředí

Studie [27] identifikovala několik klíčových výzev, které se vyskytují při implementaci automatizace testů v agilním prostředí.

2.2.4.1 Údržba a technický dluh

Studie uvádí, že tento proces je časově náročný a náchylný k chybám. S rostoucím počtem uživatelských příběhů se manuální tvorba testů (testovacích případů) stává bottleneckem, což může vést k následnému nárůstu technického dluhu, kdy se testy stávají zastaralými a všechny scénáře nemusí být pokryty, což zvyšuje riziko chyb.

2.2.4.2 Regresní testování

Jelikož agilní vývoj vyžaduje rychlé a časté změny, je klíčové zajistit, aby po každé z nich bylo co nejdříve ověřeno na regresních testech, že nedošlo k narušení stávající funkcionality. Tento problém je možno částečně řešit automatickým generováním testů přímo z požadavků, aby se minimalizoval čas mezi implementací a testováním.

2.2.4.3 Rozdíl mezi specifikací a kódem

Tato výzva souvisí s jasností požadavků na produkt. Jelikož jsou požadavky napsány v přirozeném jazyce, může docházet k nedorozuměním či nejasnostem, které mohou vést k nesprávné implementaci či chybám. To poté vede na:

- Nestabilní testovací orákulum - pokud mechanismus pro ověření výsledků není stabilní a předvídatelný, může docházet k velkému množství falešně pozitivních nálezů či křehkým testům.

- Náklady na opravu - opakovaná re-implementace funkcionalit vynucuje souběžnou údržbu testů, což výrazně snižuje návratnost (ROI) investic.
- Nízká testovatelnost - zaměření na testovatelnost od prvotních fází vývoje výrazně zvyšuje efektivitu automatizace.

2.3 Velké jazykové modely ve vývoji softwaru

2.4 Agentní systémy a autonomní testování

Závěr

3

XXXX

Přehled zkratk

X xxx

Y yyy

Uživatelská dokumentace



xxx

Bibliografie

1. WANG, Junjie et al. *Software Testing with Large Language Models: Survey, Landscape, and Vision*. 2024. Dostupné z arXiv: 2307.07221 [cs.SE].
2. DOS SANTOS, Carlos Alberto; BOUCHARD, Kevin; PETRILLO, Fabio. AI-Driven User Story Generation. In: *2024 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*. 2024, s. 1–6. Dostupné z doi: 10.1109/ACDSA59508.2024.10467677.
3. DIMA, Alina; MAASSEN, Maria Alexandra. From Waterfall to Agile software: Development models in the IT sector, 2006 to 2018. Impacts on company management. *Journal of International Studies*. 2018, roč. 11, s. 315–326. Dostupné z doi: 10.14254/2071-8330.2018/11-2/21.
4. SENARATH, Udes S. Waterfall methodology, prototyping and agile development. *no. June*. 2021.
5. ALMEIDA, Fernando. Challenges in migration from waterfall to agile environments. *World Journal of Computer Application and Technology*. 2017, roč. 5, č. 3, s. 39–49.
6. SASSA, Adrielle Cristina; ALMEIDA, Isabela Alves de; PEREIRA, Tábata Nakagomi Fernandes; OLIVEIRA, Milena Silva de. Scrum: A systematic literature review. *International Journal of Advanced Computer Science and Applications*. 2023, roč. 14, č. 4.
7. HRON, Michal; OBWEGESER, Nikolaus. Scrum in practice: an overview of Scrum adaptations. In: *Hawaii International Conference on System Sciences*. Curran Associates, Inc., 2018, s. 4496–4505.
8. SACHDEVA, Sakshi. Scrum methodology. *Int. J. Eng. Comput. Sci.* 2016, roč. 5, č. 16792, s. 16792–16800.
9. RAHARJANA, Indra Kharisma; SIAHAAN, Daniel; FATICHAH, Chastine. User Stories and Natural Language Processing: A Systematic Literature Review. *IEEE Access*. 2021, roč. 9, s. 53811–53826. Dostupné z doi: 10.1109/ACCESS.2021.3070606.

10. AMNA, Anis R.; POELS, Geert. Systematic Literature Mapping of User Story Research. *IEEE Access*. 2022, roč. 10, s. 51723–51746. Dostupné z DOI: 10.1109/ACCESS.2022.3173745.
11. POKHAREL, Prabhat; VAIDYA, Pramesh. A Study of User Story in Practice. In: *2020 International Conference on Data Analytics for Business and Industry: Way Towards a Sustainable Economy (ICDABI)*. 2020, s. 1–5. Dostupné z DOI: 10.1109/ICDABI51230.2020.9325670.
12. LUCASSEN, Garm; DALPIAZ, Fabiano; WERF, Jan Martijn EM van der; BRINK-KEMPER, Sjaak. The use and effectiveness of user stories in practice. In: *International working conference on requirements engineering: Foundation for software quality*. Springer, 2016, s. 205–222.
13. BUGLIONE, Luigi; ABRAN, Alain. Improving the User Story Agile Technique Using the INVEST Criteria. In: *2013 Joint Conference of the 23rd International Workshop on Software Measurement and the 8th International Conference on Software Process and Product Measurement*. 2013, s. 49–53. Dostupné z DOI: 10.1109/IWSM-Mensura.2013.18.
14. LI, Yishu et al. SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration. *Automated Software Engineering*. 2024, roč. 31, č. 2, s. 55.
15. FERREIRA, António MS; SILVA, Alberto Rodrigues da; PAIVA, Ana CR. Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs. In: *ENASE*. 2022, s. 477–484.
16. CHANDORKAR, Adwait; PATKAR, Nitish; DI SORBO, Andrea; NIERSTRASZ, Oscar. An Exploratory Study on the Usage of Gherkin Features in Open-Source Projects. In: *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2022, s. 1159–1166. Dostupné z DOI: 10.1109/SANER53432.2022.00134.
17. ALSHAMMARI, Abdullah. *On the use of user interface specific domain Gherkin in behaviour driven development*. 2025. Dis. pr. University of Glasgow.
18. SCHWEDT, Stefan; STRÖDER, Thomas. From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac. *arXiv preprint arXiv:2501.15181*. 2025.
19. JAMIL, Muhammad Abid; ARIF, Muhammad; ABUBAKAR, Normi Sham Awang; AHMAD, Akhlaq. Software Testing Techniques: A Literature Review. In: *2016 6th International Conference on Information and Communication Technology for The Muslim World (ICT4M)*. 2016, s. 177–182. Dostupné z DOI: 10.1109/ICT4M.2016.045.

20. HEROUT, Pavel. *Testování pro programátory*. České Budějovice: Kopp, 2015. ISBN 978-80-7232-469-1.
21. MUKHIN, Vadym et al. The Testing Mechanism for Software and Services Based on Mike Cohn's Testing Pyramid Modification. In: *2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*. IEEE, 2021, sv. 1, s. 589–595.
22. LEOTTA, Maurizio; CLERISSI, Diego; RICCA, Filippo; TONELLA, Paolo. Approaches and tools for automated end-to-end web testing. In: *Advances in Computers*. Elsevier, 2016, sv. 101, s. 193–237.
23. THE SELENIUM PROJECT. *Selenium WebDriver Documentation*. 2024. Dostupné také z: <https://www.selenium.dev/documentation/webdriver/>. Accessed: 2026-03-06.
24. ROBOT FRAMEWORK FOUNDATION. *Robot Framework User Guide*. 2024. Dostupné také z: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>. Accessed: 2026-03-07.
25. MICROSOFT. *Playwright Library Documentation*. 2024. Dostupné také z: <https://playwright.dev/docs/intro>. Accessed: 2026-03-06.
26. CYPRESS.IO. *Cypress Key Differences*. 2024. Dostupné také z: <https://docs.cypress.io/guides/overview/key-differences>. Accessed: 2026-03-07.
27. BUTT, Shimza; KHAN, Saif Ur Rehman; HUSSAIN, Shahid; WANG, Wen-Li. A conceptual model supporting decision-making for test automation in Agile-based Software Development. *Data & Knowledge Engineering*. 2023, roč. 144, s. 102111.

Seznam obrázků

2.1	Standardní šablona uživatelského příběhu.	4
2.2	Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.	5
2.3	Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu s unikátní identifikací.	6
2.4	Pyramida testování Mika Cohna (vektorový formát).	7

Seznam tabulek

2.1	INVEST principy pro tvorbu kvalitních user stories [13].	4
2.2	Druhy testů dle [20].	7

Seznam výpisů

101011000011100010 1100001
1010110001 10



11010011101101001
01100001 101
111000101011 101